

Algoritmos e Estruturas de Dados II

Força bruta e busca exaustiva

Prof. André Yoshiaki Kashiwabara

DCOMP – Universidade Federal de Sergipe

`andreyoshiaki@dcomp.ufs.br`

O que é **força bruta** — e por que a técnica mais ingênua merece uma aula



Força bruta em problemas **polinomiais**: busca, ordenação, casamento, geometria



Busca exaustiva: mochila, caixeiro viajante e atribuição — e o custo de enumerar tudo



Força bruta como **régua** e como **oráculo** de teste do algoritmo esperto

A técnica mais simples

Força bruta em problemas polinomiais

Busca exaustiva

Força bruta como régua e como oráculo

A técnica mais simples

Problema:

Diante de um problema novo, qual algoritmo escrever primeiro?

A tentação é procurar de imediato a solução esperta — e travar, ou pior, escrever algo esperto e *errado*.

Solução:

Escreva primeiro o algoritmo que segue *literalmente* o enunciado.

Ele quase sempre é lento, mas é rápido de escrever, fácil de provar correto e vira a **referência** contra a qual o algoritmo esperto será medido.

“Antes de ser rápido, seja correto. Depois torne rápido sem deixar de ser correto.”

Definição

Força bruta é a abordagem direta para resolver um problema, obtida diretamente do *enunciado* e das *definições* dos conceitos envolvidos. Nada de estrutura de dados sofisticada, nada de reaproveitar trabalho: faz-se exatamente o que o problema pede, do jeito mais óbvio.

Intuição

“Quer o maior elemento? Olhe todos.”

“Quer saber se o padrão ocorre no texto? Tente todas as posições.”

“Quer o melhor subconjunto? Examine todos os subconjuntos.”

- Força bruta é uma **decisão de projeto**, não um acidente: é a primeira linha de base, e às vezes é a resposta final.

1. Aplicabilidade universal

Serve para praticamente qualquer problema; muitas vezes é a *única* técnica disponível.

2. Linha de base

Dá um limite superior de custo: qualquer técnica melhor precisa vencer esse número.

3. Suficiente na prática

Se n é pequeno ou o código roda uma vez por mês, o simples ganha do rápido.

4. Oráculo de teste

A versão ingênua, óbvia e correta valida a versão esperta em milhares de casos.

Força bruta

Estratégia geral de projeto: resolver seguindo o enunciado ao pé da letra.

O candidato costuma ser *simples*: um índice, uma posição, um par.

caso especial

Busca exaustiva

Força bruta aplicada a problemas *combinatórios*.

O candidato é uma *estrutura*: um subconjunto, uma permutação, uma atribuição — e há exponencialmente muitos.

- Toda busca exaustiva é força bruta; nem toda força bruta é exaustiva.
- A diferença prática: força bruta polinomial é *lenta*; busca exaustiva é *inviável* já para n modesto.

Força bruta em problemas polinomiais

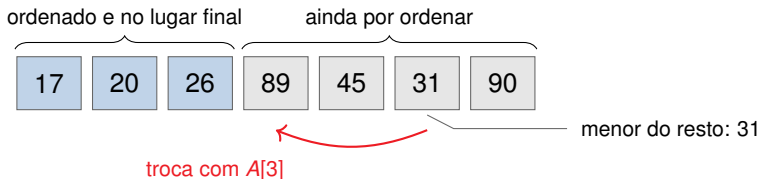
Problema: dado $A[0..n-1]$ (sem hipótese de ordenação) e um valor x , devolver um índice i com $A[i] = x$, ou -1 .

A definição de ocorrência é “existe i com $A[i] = x$ ”; a força bruta testa a definição:

```
1  int busca_sequencial(int A[], int n, int x) {  
2      for (int i = 0; i < n; i++)  
3          if (A[i] == x) return i;  
4      return -1;  
5  }
```

- Pior caso: n comparações de chave $\Rightarrow \Theta(n)$. Caso médio (busca bem-sucedida, posições equiprováveis): $(n+1)/2$.
- A busca binária é $\Theta(\log n)$, mas **exige** vetor ordenado. A força bruta não exige nada.

Definição usada: $A[0]$ é o menor de todos, $A[1]$ o menor do que sobrou, e assim por diante. O algoritmo é essa definição transcrita — daí ser força bruta (Levitin, cap. 3.1 [8]).



- Achar “o menor do que sobrou” é a **busca sequencial** do slide anterior em $A[i..n - 1]$: o custo é a soma de $n - 1$ buscas.
- Invariante: $A[0..i - 1]$ ordenado com os i menores; $n - 1$ passagens.

Na passagem i (de 0 a $n - 2$), o laço interno faz $n - 1 - i$ comparações:

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} (n - 1 - i) = \frac{n(n-1)}{2} = \Theta(n^2).$$

O que a fórmula diz

- Não há melhor caso: $\Theta(n^2)$ *sempre*, mesmo com o vetor já ordenado.
- Trocas: apenas $n - 1$, ou seja $\Theta(n)$.

Quando isso é uma virtude

Poucas trocas é bom quando *mover* um registro custa caro (registros grandes, memória lenta). O algoritmo ingênuo tem um nicho.

Cada passagem guarda só quem é o mínimo e descarta as demais comparações. O *heapsort* faz a mesma seleção num heap, que preserva esse trabalho: $\Theta(n \log n)$. Força bruta não é a estratégia; é não reaproveitar nada.

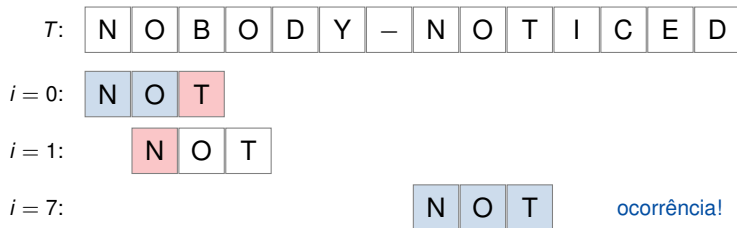
Outra leitura literal: se dois vizinhos estão fora de ordem, troque — e repita.

```
1 void bolha(int A[], int n) {  
2     for (int i = 0; i < n - 1; i++)  
3         for (int j = 0; j < n - 1 - i; j++)  
4             if (A[j+1] < A[j]) troca(&A[j], &A[j+1]);  
5 }
```

- Comparações: $n(n-1)/2 = \Theta(n^2)$; trocas no pior caso também $\Theta(n^2)$ — pior que a seleção nesse quesito.
- Uma *flag* que detecta passagem sem troca leva o melhor caso a $\Theta(n)$; o pior continua $\Theta(n^2)$.
- Lição: refinar o ingênuo raramente muda a *ordem* de crescimento. Para isso é preciso mudar de *técnica*.

Problema: dado um texto $T[0..n-1]$ e um padrão $P[0..m-1]$, achar a primeira posição i em que P ocorre em T .

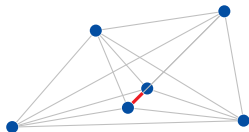
Definição de ocorrência em i : $T[i+j] = P[j]$ para todo j . A força bruta testa *todos* os $n - m + 1$ alinhamentos:



```
1  int casamento(const char *T, int n, const char *P, int m) {
2      for (int i = 0; i <= n - m; i++) {
3          int j = 0;
4          while (j < m && P[j] == T[i + j]) j++;
5          if (j == m) return i;           /* alinhamento i casou por inteiro */
6      }
7      return -1;
8  }
```

- Pior caso: $m(n - m + 1) = \Theta(nm)$, atingido por $T = \text{aaaa} \dots a$ e $P = \text{aaab}$ — cada alinhamento casa $m - 1$ letras e falha na última.
- Em texto “natural” a falha vem na primeira letra: $\Theta(n)$ na média.
- Melhorar o pior caso exige *aproveitar as comparações já feitas* (KMP, Boyer-Moore, Horspool).

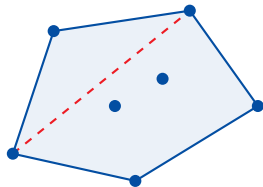
Problema: dados n pontos no plano, achar os dois de menor distância.



todos os $\binom{n}{2}$ pares; em vermelho, o mínimo

- Enumeram-se $\binom{n}{2} = \frac{n(n-1)}{2}$ pares $\Rightarrow \Theta(n^2)$.
- Detalhe de implementação: comparar d^2 em vez de d elimina todas as raízes quadradas. A ordem não muda; o relógio agradece.
- Existe algoritmo $\Theta(n \log n)$ por divisão e conquista — Unidade 2.

Problema: dados n pontos no plano, achar o menor polígono convexo que contém todos eles.



tracejado: há pontos dos dois lados
contorno: as arestas aceitas

- **Definição usada:** $p_i p_j$ é aresta do fecho quando os outros $n - 2$ pontos estão todos do *mesmo lado* da reta $p_i p_j$.
- O lado de $p = (x, y)$ é o sinal de $(x_j - x_i)(y - y_i) - (y_j - y_i)(x - x_i)$: só somas e produtos, sem trigonometria.
- $\binom{n}{2}$ pares $\times (n - 2)$ testes $\Rightarrow \Theta(n^3)$.
- Graham scan e quickhull resolvem em $\Theta(n \log n)$ — Unidade 2.

Em todos os exemplos, o algoritmo ingênuo **joga fora informação** que ele mesmo produziu:

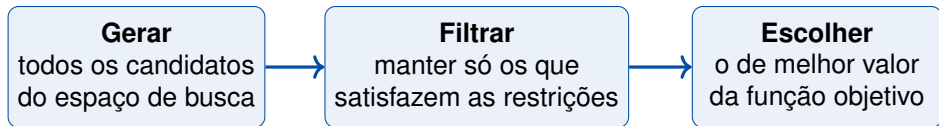
Problema	O que é desperdiçado	Técnica que aproveita
Busca em vetor ordenado	a ordem já conhecida	decrementar para conquistar
Ordenação (seleção/bolha)	comparações repetidas de pares	divisão e conquista
Casamento de padrões	o prefixo que já casou	pré-processar o padrão
Par mais próximo	vizinhança geométrica	divisão e conquista
Fecho convexo	cada aresta testada isolada	ordenar e varrer (Graham)

A pergunta de projeto do curso inteiro

Escreva a força bruta, olhe para ela e pergunte: *qual trabalho estou repetindo?* A resposta aponta a técnica a estudar.

Busca exhaustiva

Nos exemplos anteriores, o candidato era simples: um índice, um alinhamento, um par. Em muitos problemas de otimização o candidato é uma **estrutura combinatória**, e o espaço cresce exponencialmente.



Busca exaustiva

Gerar cada elemento do espaço de busca, testar viabilidade e guardar o melhor visto até agora. A corretude é trivial — *nada* é deixado de fora. O custo é que “tudo” pode ser 2^n ou $n!$.

Estrutura	Quantos	Problema típico	Decisão por elemento
Subconjuntos de n itens	2^n	mochila 0/1	incluir ou não incluir
Permutações de n itens	$n!$	caixeiro viajante	quem vem depois de quem
Atribuições $n \times n$	$n!$	problema da atribuição	qual tarefa para quem

- Hoje nos interessa **contar** e **avaliar** esses espaços.
- *Como* gerar subconjuntos e permutações sistematicamente (ordem lexicográfica, códigos de Gray, Johnson-Trotter) é o assunto da próxima semana.

Enunciado

Dados n itens com pesos $w_1, \dots, w_n$ e valores $v_1, \dots, v_n$, e uma capacidade W , escolher $S \subseteq \{1, \dots, n\}$ que

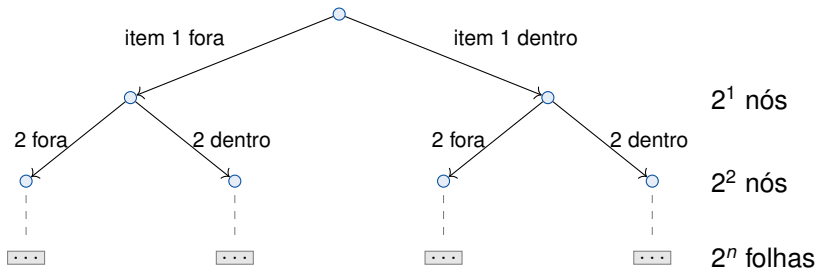
$$\text{maximize } \sum_{i \in S} v_i \quad \text{sujeito a} \quad \sum_{i \in S} w_i \leq W.$$

Cada item entra inteiro ou não entra — daí o nome “0/1”.

Item	1	2	3	4
Peso w_i	6	5	5	3
Valor v_i	30	21	20	10
Razão v_i/w_i	5,00	4,20	4,00	3,33

Capacidade $W = 10$

Mochila: a árvore de decisão tem 2^n folhas

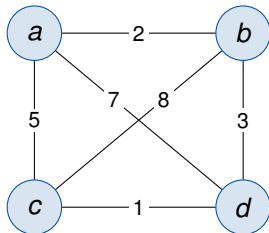


- Cada folha é um subconjunto; cada subconjunto é um número de n bits. Enumerar de 0 a $2^n - 1$ percorre o espaço inteiro.
- Guarde esta árvore: em *backtracking* (Unidade 3) ela reaparece — só que com ramos **podados** antes de chegar às folhas.

S	peso	valor	
$\emptyset$	0	0	
{1}	6	30	
{2}	5	21	
{3}	5	20	
{4}	3	10	
{1, 2}	11	51	inviável
{1, 3}	11	50	inviável
{1, 4}	9	40	

S	peso	valor	
{2, 3}	10	41	ótimo
{2, 4}	8	31	
{3, 4}	8	30	
{1, 2, 3}	16	71	inviável
{1, 2, 4}	14	61	inviável
{1, 3, 4}	14	60	inviável
{2, 3, 4}	13	51	inviável
{1, 2, 3, 4}	19	81	inviável

- Custo: $\Theta(2^n)$ subconjuntos — e $\Theta(n)$ para somar cada um, se somarmos do zero.
- Observe: o guloso por razão v_i/w_i pegaria {1, 4}, valor 40. O ótimo é {2, 3}, valor 41. **Guloso não é ótimo aqui.**



Rota (ciclo hamiltoniano)	Custo
$a-b-c-d-a$	$2 + 8 + 1 + 7 = 18$
$a-b-d-c-a$	$2 + 3 + 1 + 5 = \mathbf{11}$
$a-c-b-d-a$	$5 + 8 + 3 + 7 = 23$
$a-c-d-b-a$	$5 + 1 + 3 + 2 = \mathbf{11}$
$a-d-b-c-a$	$7 + 3 + 8 + 5 = 23$
$a-d-c-b-a$	$7 + 1 + 8 + 2 = 18$

- Fixar a cidade inicial elimina n rotações: sobram $(n - 1)!$.
- Cada rota aparece duas vezes (ida e volta): sobram $(n - 1)!/2$.

Enunciado: n pessoas, n tarefas, custo $C[i][j]$ de dar a tarefa j à pessoa i . Atribuir exatamente uma tarefa a cada pessoa minimizando o custo total. Um candidato é uma **permutação** $\langle j_1, \dots, j_n \rangle$.

	T_1	T_2	T_3	T_4
P_1	9	2	7	8
P_2	6	4	3	7
P_3	5	8	1	8
P_4	7	6	9	4

- $4! = 24$ permutações; a melhor é $\langle 2, 1, 3, 4 \rangle$ com custo $2 + 6 + 1 + 4 = 13$.
- Cuidado com a armadilha gulosa: escolher o mínimo de cada linha daria $2 + 3 + 1 + 4$, mas T_3 seria usada duas vezes — inviável.
- Aqui a busca exaustiva *não* é o destino: o método húngaro resolve em $O(n^3)$. Exponencial nem sempre é inevitável.

Quanto n cabe em um segundo?

Supondo generosamente 10^9 candidatos avaliados por segundo:

n	2^n (mochila)	tempo	$(n-1)!/2$ (TSP)	tempo
10	$1,0 \times 10^3$	$1 \mu\text{s}$	$1,8 \times 10^5$	0,2 ms
20	$1,0 \times 10^6$	1 ms	$6,1 \times 10^{16}$	1,9 anos
30	$1,1 \times 10^9$	1,1 s	$4,4 \times 10^{30}$	10^{14} anos
40	$1,1 \times 10^{12}$	18 min	—	
50	$1,1 \times 10^{15}$	13 dias	—	
60	$1,2 \times 10^{18}$	37 anos	—	

Hardware não resolve isto

Um computador $1000\times$ mais rápido acrescenta cerca de 10 ao n viável em 2^n — e apenas 2 ou 3 ao n viável em $n!$. O ganho é aditivo; o custo é multiplicativo.

O que virá no curso

- **Podar** a árvore sem perder o ótimo: backtracking e *branch and bound* (Unidade 3).
- **Reaproveitar subproblemas:** programação dinâmica — mochila em $O(nW)$ (Unidade 2).
- **Trocar ótimo por rapidez:** heurísticas e algoritmos de aproximação.

O que *não* virá

Para mochila 0/1, TSP e outros problemas NP-difíceis, ninguém conhece algoritmo polinomial — e há boas razões para acreditar que não existe (Unidade 3).

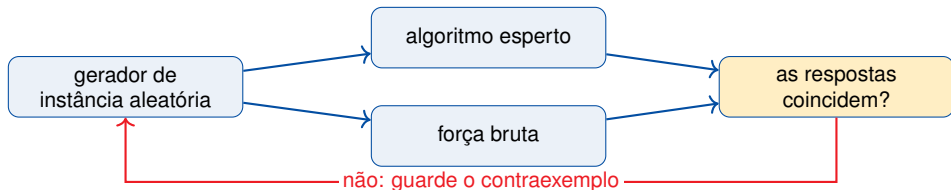
Nesses casos, a busca exaustiva podada é a base de tudo o que realmente se usa.

Instância pequena, uso ocasional, resposta exata obrigatória: **use força bruta e durma tranquilo.**

**Força bruta como régua e como
oráculo**

Problema	Força bruta	Melhor conhecido	Técnica
Busca em vetor ordenado	$\Theta(n)$	$\Theta(\log n)$	decrementar e conquistar
Ordenação	$\Theta(n^2)$	$\Theta(n \log n)$	divisão e conquista
Casamento de padrões	$\Theta(nm)$	$\Theta(n + m)$	pré-processamento
Par mais próximo	$\Theta(n^2)$	$\Theta(n \log n)$	divisão e conquista
Mochila 0/1	$\Theta(2^n)$	$O(nW)$ pseudopolin.	programação dinâmica
Atribuição	$\Theta(n!)$	$O(n^3)$	método húngaro
Caixeiro viajante	$\Theta(n!)$	$O(2^n n^2)$	PD / <i>branch and bound</i>

- Esta tabela é o mapa da disciplina. Cada linha vira uma aula.
- Note a última: mesmo o “melhor conhecido” do TSP é exponencial.



- A força bruta é lenta, mas *obviamente correta*: é o padrão-ouro.
- Rode os dois com $n \leq 10$ e milhares de instâncias aleatórias.
- Divergência $\Rightarrow$ uma entrada concreta para depurar.
- Teste mostra a presença de erro, não a ausência: não substitui a prova.

O que aprendemos

- Força bruta é resolver *diretamente pelo enunciado*: universal, fácil de provar correta, quase sempre lenta.
- Em problemas polinomiais ela custa $\Theta(n)$, $\Theta(n^2)$ ou $\Theta(nm)$; o desperdício visível aponta a técnica que vem depois.
- Busca exaustiva é força bruta sobre espaços combinatórios: 2^n subconjuntos, $n!$ permutações. Inviável já para n modesto — e hardware não salva.
- A versão ingênua serve como **régua** de comparação e como **oráculo** de teste da versão esperta.

Escreva a força bruta primeiro. Depois pergunte que trabalho ela repete.

No tutorial

- Casamento ingênuo contando comparações no pior caso.
- Mochila exaustiva por máscara de bits.
- TSP exaustivo e a medição do salto fatorial.
- Desafio: usar a mochila exaustiva como oráculo para flagrar o erro do algoritmo guloso.

A seguir

Seleção e estatística de ordem: como achar o k -ésimo menor elemento sem ordenar o vetor (quickselect, mediana).

Depois: geração combinatória — como percorrer sistematicamente os 2^n subconjuntos e as $n!$ permutações que hoje só contamos.



Jon Bentley.

Programming Pearls.

Addison-Wesley, 2 edition, 2000.



Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein.

Introduction to Algorithms.

MIT Press, 3 edition, 2009.



Michael R. Garey and David S. Johnson.

Computers and Intractability: A Guide to the Theory of NP-Completeness.

W. H. Freeman, 1979.



David Gries.

The Science of Programming.

Springer-Verlag, 1981.



Michael Held and Richard M. Karp.

A dynamic programming approach to sequencing problems.

Journal of the Society for Industrial and Applied Mathematics, 10(1):196–210, 1962.



C. A. R. Hoare.

Algorithm 64: Quicksort.

Communications of the ACM, 4(7):321, 1961.



C. A. R. Hoare.

An axiomatic basis for computer programming.

Communications of the ACM, 12(10):576–580, 1969.



Anany Levitin.

Introduction to the Design and Analysis of Algorithms.

Pearson, 3 edition, 2012.



Udi Manber.

Introduction to Algorithms: A Creative Approach.

Addison-Wesley, 1989.



Robert Sedgewick and Kevin Wayne.

Algorithms.

Addison-Wesley, 4 edition, 2011.



Nivio Ziviani.

Projeto de Algoritmos: com implementações em Pascal e C.

Cengage Learning, 3 edition, 2010.